

ZeroDDS

DDS-TS 1.0

DDS-TS

TypeScript Platform-Specific Model for OMG IDL 4.2

Version **zerodds/v1.0**

Status: Final

Date: 2026-04-29

Editors: Sandra Keßler

Copyright © 2026 ZeroDDS Project. All rights reserved.

This specification is published under the ZeroDDS Vendor Specification Licence.

Contents

Status of this Document	5
1 Scope	6
1.1 Purpose	6
1.2 Out of Scope	6
2 Conformance	7
2.1 IDL-Codegen Profile	7
2.2 Descriptor-Runtime Profile	7
2.3 Operations Profile	8
2.4 WASM-Bindings Profile	8
3 Normative References	10
4 Terms and Definitions	11
5 Symbols and Abbreviations	12
6 Overview (Informative)	13
6.1 Mapping Philosophy	13
6.2 Runtime Reflection Strategy	13
6.3 Toolchain	14
6.4 Code-Style Toggle	14
7 Type Mapping (Normative)	15
7.1 Module Mapping	15
7.2 Constant Mapping	15
7.3 Primitive Types	16
7.3.1 Bounded Strings	17
7.3.2 Branded Character Types	17
7.3.3 <code>long double</code> Handling	18
7.3.4 <code>any</code> Mapping	19
7.4 Struct Mapping	19
7.4.1 Uniform Interface Mapping	19
7.4.2 Type Descriptor Side-Table	20
7.4.3 Exception Mapping	21
7.5 Union Mapping	22
7.5.1 Discriminated-Union Form	22
7.5.2 Multiple Labels Per Case	23
7.5.3 Union Descriptor Side-Table	23

7.5.4	Implicit Default	23
7.6	Enum Mapping	24
7.6.1	Default Form	24
7.6.2	@value(N)	24
7.6.3	@bit_bound(N)	25
7.6.4	Enum Descriptor Side-Table	25
7.7	Bitset Mapping	25
7.7.1	Bitset Descriptor Side-Table	26
7.8	Bitmask Mapping	26
7.8.1	Bitmask Descriptor Side-Table	27
7.8.2	Position Annotation	27
7.9	Sequence and Array Mapping	28
7.9.1	Bounded and Unbounded Sequences	28
7.9.2	Fixed-Size Arrays	28
7.9.3	Multi-Dimensional Arrays	28
7.9.4	Element Mutability	29
7.9.5	Empty Sequences	29
7.9.6	Maps	29
7.10	Typedef Mapping	30
7.10.1	Simple Type Aliases	30
7.10.2	Sequence Aliases	30
7.10.3	Array Aliases	31
7.10.4	Recursive Resolution	31
7.10.5	Forward References	31
7.10.6	Typedef Descriptor Side-Table	31
7.10.7	@bit_bound on Integer Typedefs	31
7.11	Optional and Default Values	32
7.11.1	Optional Fields	32
7.11.2	Encoding of Absent Values (Informative)	33
7.11.3	Default Values	33
7.12	Annotation Mapping	34
7.12.1	Mapping Surface	34
7.12.2	Mapping Table	34
7.12.3	Annotation Resolution Order	35
7.12.4	TSDoc Rendering	36
8	Descriptor Runtime (Normative)	37
8.1	Extensibility Kind	37
8.2	Type References	37
8.3	Type and Member Descriptors	38
8.4	Boxed any	39
8.5	Branded Character Helpers	39
8.6	LongDouble	40
8.7	Reflection API	40
9	Verbatim Code-Generation Hook	43
9.1	Purpose	43

9.2	Annotation Form	43
9.3	Placement Kinds	43
9.4	Quoting and Escaping	44
9.5	Multi-Line Text	44
9.6	Multiple Annotations on One Declaration	44
9.7	Reference Implementation Footprint	44
10	Code Style (Informative)	46
10.1	Identifier Casing	46
10.2	Output Format	46
10.3	File Layout	46
11	Interface and Operation Mapping (Normative)	47
11.1	Mapping Surface	47
11.2	Operation Mapping	47
11.3	Attribute Mapping	49
11.4	Exception Mapping	49
11.5	Service Descriptor	49
11.6	Inheritance	50
11.7	Forward Declarations	51
Annex A	(informative): IDL Examples and Generated Output	52
Annex B	(normative): Compliance Test Suite	54
Annex C	(normative): WASM-Bindings Profile	58
Annex D	(normative): Diagnostic Codes	64

Status of this Document

This specification (version **zeroddsv1.0**) is published by the ZeroDDS Project. It defines a Platform-Specific Model (PSM) that maps OMG IDL 4.2 to TypeScript. Its document organisation follows the OMG language PSMs (IDL-to-C++, IDL-to-Java, IDL-to-C#); its type-system mapping deliberately diverges, using TypeScript's structural interfaces rather than the nominal classes of those PSMs (§6.1).

This document is not an OMG document and is not on any OMG standardisation track. The conformance surface is nevertheless designed for cross-vendor interoperability: the brand schemas (§11.7, §7.3.2), exported symbol names, operation/exception encodings (Chapter 11), and diagnostic codes (Annex 11.7) are vendor-neutral.

TypeScript is not an OMG or ISO standard; its prose specification (Microsoft, last edition 1.8 / 2016) is no longer maintained. The de-facto reference is the **tsc** compiler distributed as the **typescript** npm package. A conformant generator output SHALL compile cleanly under **tsc -strict** for any **typescript** package release at major version 5 (i.e. 5.0.0 up to but not including 6.0.0). Runtime semantics are inherited from ECMA-262 (14th edition or later).

The terms **MUST**, **MUST NOT**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **MAY**, **MAY NOT**, **REQUIRED**, **RECOMMENDED**, and **OPTIONAL** are to be interpreted as described in IETF RFC 2119 and RFC 8174.

1 Scope

1.1 Purpose

This specification defines a TypeScript Platform-Specific Model (PSM) for the OMG Interface Definition Language version 4.2, with extensions for the Extensible and Dynamic Topic Types for DDS (X-Types) version 1.3 annotations.

The mapping enables IDL types defined for DDS topics to be generated as TypeScript declarations and consumed in browser-side or Node.js-side TypeScript code without loss of semantic information.

1.2 Out of Scope

The following items are explicitly outside the scope of this specification:

- TypeScript runtime support beyond the descriptor-runtime library defined in Chapter 8.
- Redefinition of any DDS wire-encoding format. Wire encoding is governed by OMG X-Types (XCDR1, XCDR2); this document references those rules but does not redefine them.
- Bindings to a specific JavaScript runtime: this specification produces ESM source compilable under `tsc -strict`.
- DDS-RPC wire protocol; covered by OMG DDS-RPC. This PSM only specifies the TypeScript surface through which RPC client and server code is generated (Chapter 11).

The mapping covers the full structural surface of OMG IDL 4.2: modules, free-standing constants, primitive types, constructed types (struct, union, enum, bitset, bitmask, sequence, array, typedef, exception), and the **interface** construct with its operations and attributes.

2 Conformance

This specification defines four conformance profiles. A conformance claim **MUST** cite this specification by full identifier (DDS-TS, zerodds/v1.0 or later) and enumerate every profile to which conformance is claimed.

2.1 IDL-Codegen Profile

Conformance: IDL-Codegen Profile

An implementation conforms to the IDL-Codegen Profile if and only if all of the following are true:

1. It implements the type mapping defined in Chapter 7 for every primitive, constructed, and template type of OMG IDL 4.2.
2. It implements the annotation mapping defined in § 7.12 for every X-Types 1.3 annotation listed in that section.
3. For every IDL type it emits, it additionally emits a *Type Descriptor* (Chapter 8) and a corresponding type-guard function, per § 7.4.
4. It produces output that is valid TypeScript $\geq$ 5.0 source code, compilable with `tsc -strict` without the `experimentalDecorators` or `emitDecoratorMetadata` compiler options.
5. It satisfies the conformance test cases in Annex B, § B.1.

2.2 Descriptor-Runtime Profile

Conformance: Descriptor-Runtime Profile

An implementation conforms to the Descriptor-Runtime Profile if and only if all of the following are true:

1. It provides the descriptor types `DdsTypeDescriptor`, `DdsMemberDescriptor`, `DdsTypeRef`, `ExtensibilityKind`, `DdsAny`, with the structural shapes defined in Chapter 8.
2. It provides the reflection functions `getKey`, `getTopic`, `registerType`, `lookupType`,

`unboxAny`, `boxAny` with the semantics defined in Chapter 8.

3. It provides the runtime helpers `makeChar` and `makeWChar` per § 7.3.2.
4. It satisfies the conformance test cases in Annex B, § B.2.

2.3 Operations Profile

Conformance: Operations Profile

An implementation conforms to the Operations Profile if and only if all of the following are true:

1. It conforms to the IDL-Codegen Profile (§ 2.1) and to the Descriptor-Runtime Profile (§ 2.2).
2. It implements the **interface**, operation, attribute, and exception mapping defined in Chapter 11.
3. For every IDL **interface** it emits, it additionally emits a corresponding *Service Descriptor* (§ 11.5).
4. It provides the operations descriptor types `ServiceDescriptor`, `OperationDescriptor`, `OperationParameterDescriptor`, `AttributeDescriptor`, `ParameterMode`, and the runtime helper `isOneOf`, with the structural shapes defined in § 11.5.
5. It satisfies the conformance test cases in Annex B, § B.3.

The Operations Profile is independent of the WASM-Bindings Profile; an implementation MAY claim either, neither, or both.

2.4 WASM-Bindings Profile

Conformance: WASM-Bindings Profile

An implementation conforms to the WASM-Bindings Profile if and only if all of the following are true:

1. It conforms to the IDL-Codegen Profile (§ 2.1) and to the Descriptor-Runtime Profile (§ 2.2).
2. It exposes the DDS DCPS data-path operations enumerated in Annex C, § C.1, as TypeScript-callable functions backed by a WebAssembly module compiled from a DDS implementation (e.g. the ZeroDDS Rust core).
3. It serialises and deserialises samples using the XCDR2 wire format on both sides of

the WASM boundary; the binding layer MUST NOT expose JavaScript-native object representations as the canonical wire form.

4. It satisfies the conformance test cases in Annex C, § C.2 (round-trip publish/subscribe through the WASM boundary).

The design and operational semantics of the WASM-Bindings Profile are normatively specified in Annex C.

3 Normative References

Identifier	Version	Title
OMG IDL	4.2	Interface Definition Language
OMG X-Types	1.3	Extensible and Dynamic Topic Types for DDS
OMG DDS	1.4	Data Distribution Service for Real-Time Systems
OMG DDS-RPC	1.0	Remote Procedure Call over DDS
OMG IDL4-Cpp	1.0	IDL to C++ Mapping (OMG language PSM)
OMG IDL4-Csharp	1.0	IDL to C# Mapping (OMG language PSM)
OMG IDL4-Java	1.0	IDL to Java Mapping (OMG language PSM)
ECMA-262	14th ed.	ECMAScript Language Specification (TC39)
Microsoft <code>tsc</code>	5.0+	TypeScript compiler behaviour (<code>typescript</code> npm package, github.com/microsoft/TypeScript); see §3
IETF RFC 2119	—	Key words for use in RFCs
IETF RFC 8174	—	Ambiguity of Uppercase vs Lowercase

4 Terms and Definitions

IDL OMG Interface Definition Language as defined in *OMG IDL* 4.2.

Codegen An implementation conforming to the IDL-Codegen Profile (§ 2.1).

Type Descriptor

A side-table value of TypeScript type `DdsTypeDescriptor<T>` that captures the metadata of an IDL type (name, extensibility, members, topic binding, type-guard); see Chapter 8.

Service Descriptor

A side-table value of TypeScript type `ServiceDescriptor<TClient, THandler>` that captures the metadata of an IDL **interface** (name, operations, attributes, exception sets, inheritance); see Chapter 11.

ESM ECMAScript Module syntax: **export** and **import** statements as defined in ECMA-262.

5 Symbols and Abbreviations

DDS Data Distribution Service

ESM ECMAScript Module

IDL Interface Definition Language

PSM Platform-Specific Model

TS TypeScript

WASM WebAssembly

XCDR2 Extended Common Data Representation, version 2

6 Overview (Informative)

6.1 Mapping Philosophy

The DDS-TS mapping uses TypeScript's structural typing rather than the nominal class-based mappings of the OMG C++, Java, and C# PSMs. Every IDL data type is mapped to an **interface**; annotation metadata is exposed via a separately-emitted *Type Descriptor* (Chapter 8). The same approach applies to IDL **interface** mappings (Chapter 11), which produce paired client/handler interfaces plus a *Service Descriptor*. No TypeScript class is emitted by this PSM in either path.

6.2 Runtime Reflection Strategy

TypeScript interfaces are erased during compilation; a plain object literal carries no JavaScript-level type identity. The PSM closes this gap with first-class side-table values:

- Every IDL type emits an `export const <Type>Type: DdsTypeDescriptor<T>` constant, registered with a process-wide `DdsTypeRegistry` (§ 8.7).
- Every IDL interface emits an `export const <Iface>Service: ServiceDescriptor<C, H>` constant.
- Every IDL **any** value crosses the TypeScript surface as a `DdsAny{ typeId, value }` carrier whose `typeId` indexes the registry.

This is a deliberate parallel to the runtime type system that OMG X-Types 1.3 itself defines for cross-vendor type discovery: the `TypeObject` / `MinimalTypeObject` structures of OMG X-Types 1.3 §7.3 carry exactly the same metadata (extensibility, member ids, key membership, type references) as first-class values that DDS implementations exchange over the Type-Lookup service. The DDS-TS `DdsTypeDescriptor` is the TypeScript-side projection of that runtime metadata: the content set is intentionally close to `TypeObject`, so that an implementation MAY construct a `TypeObject` from a `DdsTypeDescriptor` (and vice versa) without information loss.

The side-table approach is necessary because TypeScript has no Plain-Object-Reflection mechanism comparable to Java's `Class<T>` or C#'s `System.Type`: structural interface-based code carries no runtime type identity, so the identity must be carried alongside the data as a first-class value. This PSM standardises the shape of that sidecar.

6.3 Toolchain

The reference implementation in `crates/idl-ts/` produces `.ts` source files compatible with TypeScript ≥ 5.0 . The generated source uses ECMAScript-Module (ESM) syntax exclusively.

6.4 Code-Style Toggle

By default, IDL identifier names are preserved verbatim (`snake_case` to `snake_case`, `Camel-Case` to `CamelCase`). An optional *camelCase-toggle* configuration (Chapter 10) re-cases `snake_case` IDL fields to `camelCase` TypeScript fields.

7 Type Mapping (Normative)

7.1 Module Mapping

An IDL `module` SHALL be mapped to a TypeScript `export namespace` declaration of the same name. Nested `module` declarations SHALL produce nested `export namespace` declarations preserving the IDL nesting order.

```
module Vehicle {  
  module Sensor {  
    struct Reading { double value; };  
  };  
  struct Car { string vin; };  
};
```

maps to

```
export namespace Vehicle {  
  export namespace Sensor {  
    export interface Reading { value: number; }  
  }  
  export interface Car { vin: string; }  
}
```

A reference to a qualified IDL name (`::Vehicle::Sensor::Reading`) SHALL be emitted as the corresponding dotted TypeScript path (`Vehicle.Sensor.Reading`). Implementations MAY additionally produce one ECMAScript module file per top-level IDL module under a `-module-per-file` configuration option; the namespace structure inside each file SHALL remain as specified above.

A re-opening of the same module name across compilation units SHALL be honoured: TypeScript `export namespace` permits declaration merging and the code generator SHALL rely on that mechanism rather than emitting a renamed namespace.

7.2 Constant Mapping

A free-standing IDL constant declaration of the form `const <type> <name> = <expr>;` SHALL be mapped to a TypeScript `export const <name>: T' = <expr'>;` declaration, where `T'` is the TypeScript-mapped form of `<type>` per §7.3 and `<expr'>` is the const-evaluated literal in TypeScript syntax.

```
const long    MAX_RETRIES    = 5;
const double  PI             = 3.14159265358979;
const string  SERVICE_NAME  = "telemetry";
const boolean DEBUG_ENABLED = FALSE;
```

maps to

```
export const MAX_RETRIES:    number    = 5;
export const PI:             number    = 3.14159265358979;
export const SERVICE_NAME:  string    = "telemetry";
export const DEBUG_ENABLED: boolean    = false;
```

The implementation SHALL const-evaluate the right-hand-side expression at code-generation time. Decimal, hexadecimal, octal, and binary integer-literal forms MUST be supported, as well as character escapes inside string literals (per IDL §7.2.6.5). Constants of type `long` `long` or `unsigned long` `long` SHALL be emitted with a trailing `n` BigInt literal: `export const X: bigint = 9007199254740993n;`.

A constant whose right-hand side references another constant SHALL be emitted using the referenced TypeScript identifier; the implementation SHALL resolve the dependency in declaration order and SHALL reject cyclic references at parse time. Constants declared inside an IDL module SHALL appear inside the corresponding TypeScript `export namespace` (§7.1).

7.3 Primitive Types

IDL Type	TypeScript Type	Notes
<code>boolean</code>	<code>boolean</code>	
<code>char</code>	<code>Char</code>	branded type, see §7.3.2
<code>wchar</code>	<code>WChar</code>	branded type, see §7.3.2
<code>octet</code>	<code>number</code>	0..255
<code>short</code> / <code>int16</code>	<code>number</code>	
<code>unsigned short</code> / <code>uint16</code>	<code>number</code>	
<code>long</code> / <code>int32</code>	<code>number</code>	
<code>unsigned long</code> / <code>uint32</code>	<code>number</code>	
<code>long long</code> / <code>int64</code>	<code>bigint</code>	JS number is 53-bit exact
<code>unsigned long</code>	<code>bigint</code>	
<code>long</code> / <code>uint64</code>		
<code>float</code>	<code>number</code>	IEEE 754 binary32
<code>double</code>	<code>number</code>	IEEE 754 binary64
<code>long double</code>	<code>LongDouble</code>	opaque 16-byte carrier; see §7.3.3

IDL Type	TypeScript Type	Notes
string / wstring	string	bounded form: see § 7.3.1
any	DdsAny	boxed value with type-id; see § 7.3.4

7.3.1 Bounded Strings

A bounded IDL `string<N>` or `wstring<N>` SHALL produce a TypeScript `string` property as above, plus an exported numeric constant `<Container>_<Field>_BOUND` emitted alongside the type declaration carrying `N`, in the same style used for sequences (§ 7.9) and maps (§ 7.9.6). The bound SHOULD be checked at the runtime-validation layer; the TypeScript `string` type cannot enforce a maximum length structurally. Standalone bounded-string typedefs SHALL emit `<Alias>_BOUND` analogously.

7.3.2 Branded Character Types

IDL `char` and `wchar` hold exactly one character; TypeScript cannot express a fixed character count structurally. The mapping SHALL therefore use branded aliases with string-literal tags (chosen so that brands are stable across vendor and module boundaries):

```
export type Char = string & { readonly __dds_brand: "char" };
export type WChar = string & { readonly __dds_brand: "wchar" };
```

The property name `__dds_brand` and the tag values `"char"`, `"wchar"` are normative. Construction SHALL use the factory helpers below; runtime enforcement is the responsibility of the boundary layer.

makeChar. The IDL `char` is an 8-bit quantity in the range 0..255 holding a single ISO 8859-1 character (OMG IDL 4.2 §7.2.6.2). The helper SHALL accept a value if and only if its UTF-16 code-unit length is exactly one and the single Unicode code point lies in the inclusive range U+0000..U+00FF; it SHALL throw a `RangeError` otherwise.

```
export function makeChar(s: string): Char {
  if (s.length !== 1 || s.codePointAt(0)! > 0xFF) {
    throw new RangeError(
      "Char requires single ISO 8859-1 code point");
  }
  return s as Char;
}
```

makeWChar. The IDL `wchar` is a full Unicode scalar (OMG IDL 4.2 §7.2.6.2): the value space is U+0000..U+10FFFF excluding the surrogate range U+D800..U+DFFF. Because JavaScript strings are UTF-16 code-unit sequences, scalars above the BMP occupy two code units (a surrogate pair); length-checking MUST use code-point count, not code-unit count.

```
export function makeWChar(s: string): WChar {
  // String iterator yields code points, not code units.
  const cps = [...s];
  if (cps.length !== 1) {
    throw new RangeError(
      "WChar requires exactly one Unicode scalar");
  }
  const cp = s.codePointAt(0)!;
  if (cp >= 0xD800 && cp <= 0xDFFF) {
    throw new RangeError(
      "WChar must not be an unpaired surrogate");
  }
  return s as WChar;
}
```

Implementations MUST NOT validate `wchar` via `string.length === 1`, which would reject every astral-plane character.

7.3.3 long double Handling

TypeScript `number` is IEEE 754 binary64 (ECMA-262 §6.1.6.1) and cannot represent long double without precision loss. IDL long double SHALL therefore be mapped to the opaque carrier `LongDouble`:

```
export interface LongDouble {
  readonly __dds_brand: "long_double";
  readonly bytes: Uint8Array; // length === 16
}

export function makeLongDouble(bytes: Uint8Array): LongDouble {
  if (bytes.length !== 16) {
    throw new RangeError("LongDouble requires 16 bytes");
  }
  return { __dds_brand: "long_double", bytes };
}
```

The branded shape, the brand tag `"long_double"`, and `bytes.length === 16` are normative. The byte order *within* `bytes` is implementation-defined; wire encoding across a DDS boundary is governed by OMG X-Types XCDR2 §7.4.3 and is out of scope here. Arithmetic on `LongDouble` is outside the scope of this PSM.

7.3.4 any Mapping

The IDL **any** type carries a value of any IDL type together with an explicit run-time type identity (OMG IDL 4.2 §7.4.14). Because TypeScript **interface** declarations have no runtime representation, an *unwrapped* payload object provides no basis for run-time type discovery. The code generator therefore SHALL map **any** to the boxed type **DdsAny**, defined in Chapter 8:

```
export interface DdsAny {
  readonly typeId: string;           // qualified IDL name
  readonly value: unknown;          // the boxed payload
}
```

Producers SHALL construct a **DdsAny** via `boxAny(descriptor, value)`; consumers SHALL narrow via `unboxAny(any, descriptor)`, which checks `any.typeId` against the descriptor name and applies the descriptor’s type-guard before returning the typed value.

Static unknowability. The runtime type identity of a **DdsAny** value is by construction not statically known at the TypeScript surface; the corresponding `DdsTypeRef.kind = "any"` carries no **name** or descriptor reference. A consumer that walks a descriptor graph and encounters an **any-kind** type-ref SHALL defer type-resolution to the boxed value’s `typeId`, typically by passing it to `lookupType(any.typeId)`. Implementations SHALL NOT map **any** to TypeScript **any** or **unknown**; either choice would defeat the type-identity guarantee that the IDL **any** construct requires.

7.4 Struct Mapping

7.4.1 Uniform Interface Mapping

An IDL **struct** declaration SHALL be mapped to a TypeScript **export interface** declaration of the same name. Each IDL field SHALL be mapped to a property of the interface whose TypeScript type is the mapped form of the IDL field type. This rule is independent of any annotation attached to the struct or its fields; annotations SHALL influence the emitted Type Descriptor (§7.4.2) and TSDoc tags, never the shape of the interface itself.

Implementations SHALL NOT promote a struct to a TypeScript **class** on the basis of annotations; this PSM does not emit any TypeScript class for IDL data types. The previous “class-promotion” design is explicitly disallowed.

```
struct Point { long x; long y; };
```

maps to

```
export interface Point {
  x: number;
```

```
y: number;
}
```

The same shape SHALL be emitted for an annotated struct; only the side-table descriptor changes.

```
@final
struct Point {
  @key long x;
  @key long y;
};
```

maps to

```
export interface Point {
  x: number;
  y: number;
}
```

plus the descriptor of §7.4.2.

7.4.2 Type Descriptor Side-Table

For every IDL `struct` the code generator SHALL additionally emit:

1. A constant of type `DdsTypeDescriptor<T>` (Chapter 8) named `<TypeName>Type`, capturing the struct's name, extensibility kind, topic binding (if any), nested flag, and the ordered list of `DdsMemberDescriptor` entries.
2. A type-guard function named `is<TypeName>` of TypeScript type `(v: unknown) => v is <TypeName>`, whose body SHALL verify the presence and type of every required member of `<TypeName>`. Optional members SHALL NOT be required by the guard. The guard SHALL NOT validate `@unit`, `@min`, or `@max` bounds; those are TSDoc tags only.
3. A side-effect-free registration call `registerType(<TypeName>Type)` that adds the descriptor to the global `DdsTypeRegistry` (used by `unboxAny` when the boxed `typeId` matches).

```
@final
@topic("Pose")
struct Pose {
  @key string<32> frame_id;
  @id(1) double x;
  @id(2) double y;
};
```

maps to (interface plus side-table)

```
export interface Pose {
  frame_id: string;
  x:        number;
  y:        number;
}

export const PoseType: DdsTypeDescriptor<Pose> = {
  kind:          "struct",
  name:          "Pose",
  extensibility: "final",
  nested:        false,
  topic:         "Pose",
  fields: [
    { name: "frame_id", id: 0, type: { kind: "string", bound: 32,
      wide: false }, key: true,  optional: false, mustUnderstand:
      false },
    { name: "x",          id: 1, type: { kind: "primitive", name: "
      double" },          key: false, optional: false, mustUnderstand:
      false },
    { name: "y",          id: 2, type: { kind: "primitive", name: "
      double" },          key: false, optional: false, mustUnderstand:
      false },
  ],
  typeGuard: isPose,
};

export function isPose(v: unknown): v is Pose {
  return typeof v === "object" && v !== null
    && typeof (v as any).frame_id === "string"
    && typeof (v as any).x === "number"
    && typeof (v as any).y === "number";
}

registerType(PoseType);
```

7.4.3 Exception Mapping

An IDL exception declaration SHALL be mapped identically to a struct (§ 7.4.1§ 7.4.2), with two additions:

1. The interface declaration SHALL extend a marker type `DdsException` (exported from `@zerodds/types`, structurally `{ readonly __dds_exception: true }`) so that TypeScript can distinguish exception payloads from ordinary structs at compile time.
2. The descriptor's `kind` field SHALL be "exception" rather than "struct".

For raising an IDL exception across an Operations Profile boundary, see § 11.4.

7.5 Union Mapping

7.5.1 Discriminated-Union Form

An IDL **union** SHALL be mapped to a TypeScript discriminated-union type aliased by the IDL union name. Each branch (**<case>** grammar (130) of OMG IDL 4.2) SHALL be a distinct anonymous-interface arm of the union, carrying the discriminator value and the case-member typed property.

The discriminator property SHALL be named **discriminator**. Its TypeScript type SHALL be literal-narrowed to the case label, not to the discriminator's broad mapped type:

- Discriminator type **integer** or **octet**: each arm's **discriminator** SHALL be the literal numeric case label (e.g. 1, 2).
- Discriminator type **boolean**: **true** or **false**.
- Discriminator type **char**: a literal one-character **Char**; the helper `makeChar(...)` SHOULD be used at construction.
- Discriminator type **enum**: the corresponding `<EnumName>.<Member>` literal.

If the IDL union contains a **default** case (grammar (132) **<default_case>**), the implementation SHALL emit a final arm whose **discriminator** property carries the TypeScript-mapped form of the discriminator type *minus* the union of explicit case labels:

```
discriminator: Exclude<DiscType, ExplicitLabels>;
```

Compile-time exclusivity is partial. TypeScript's `Exclude<T, U>` narrows literal-typed unions but does not narrow primitive types: `Exclude<number, 1 | 2>` remains `number`. The compile-time exclusivity of the default arm therefore holds for **enum** and **boolean** discriminators (where labels are literal subtypes of the discriminator's TypeScript type) and *not* for integer or **octet** discriminators. For the latter, the default arm is structurally a **number** arm; runtime discriminator-matching enforces case selection. Implementations MAY additionally tag the default arm with a brand property (e.g. `readonly __default: true`) for narrower compile-time discrimination, but this brand is not normative.

If the IDL union has no **default** case, the resulting TypeScript type is closed for **enum** and **boolean** discriminators; for integer discriminators the closure is runtime-only.

The case labels SHALL be distinct; an IDL union with two **case**-clauses sharing a label is rejected by the IDL parser (OMG IDL 4.2 §7.4.1.4.4.5) and never reaches the code generator.

```
enum Tag { TAG_A, TAG_B };
union Variant switch (Tag) {
  case TAG_A:      long    i;
  case TAG_B:      string  s;
  default:         octet   raw;
};
```

maps to

```
export type Variant =
  | { discriminator: Tag.TAG_A; i: number }
  | { discriminator: Tag.TAG_B; s: string }
  | { discriminator: Exclude<Tag, Tag.TAG_A | Tag.TAG_B>; raw:
      number };

```

7.5.2 Multiple Labels Per Case

A case body MAY list more than one label (`case L1: case L2: T m;`); the implementation SHALL expand each label into a separate arm of the TypeScript discriminated union, each carrying the same case member.

7.5.3 Union Descriptor Side-Table

For every IDL union the code generator SHALL emit a constant of type `DdsTypeDescriptor<T>` with `kind: "union"`, plus a type-guard `is<UnionName>`. The descriptor's `fields` array SHALL list one `DdsMemberDescriptor` per case member; an additional synthetic descriptor field named `discriminator` with `id: 0xFFFFFFFF` SHALL appear first and carry the discriminator's `DdsTypeRef`. The reserved discriminator id (`0xFFFFFFFF`) follows the OMG XTypes convention for synthetic member-ids and MUST NOT be used by any user-declared member; the IDL parser SHALL reject any `@id(0xFFFFFFFF)` (§7.12). Each non-discriminator descriptor entry SHALL additionally carry the `labels` property listing the matching case labels:

```
readonly labels?: ReadonlyArray<number | string | boolean>;

```

The default case (when present) SHALL have `labels: undefined` and the descriptor SHALL expose a flag `hasDefault: true`.

7.5.4 Implicit Default

If the IDL union does not list a `default` case and the discriminator is not exhaustively covered by the listed cases (OMG X-Types 1.3 §7.2.2.4.4.4.5), the descriptor SHALL mark this with `hasImplicitDefault: true`; an implementation SHALL treat construction with an out-of-range discriminator at the runtime layer as a deserialisation error, signalled by the runtime, not a TypeScript compile-time error.

7.6 Enum Mapping

7.6.1 Default Form

An IDL `enum` SHALL be mapped to a TypeScript `const` object plus a string-literal-union type alias. The implementation SHALL NOT emit a TypeScript `enum` declaration: TypeScript `enum` produces a nominal type that is not assignment-compatible with plain string literals (forcing all callers to import the enum) and behaves inconsistently under `-isolatedModules`, which breaks single-file transpilers (esbuild, swc).

```
enum Color { RED, GREEN, BLUE };
```

maps to

```
export const Color = {
  RED: "RED",
  GREEN: "GREEN",
  BLUE: "BLUE",
} as const;
export type Color = (typeof Color)[keyof typeof Color];
```

The aliased type is the union of the literal-string values ("RED" | "GREEN" | "BLUE"); the `const` object provides a value-level handle for switch statements and discriminator labels (§7.5).

7.6.2 @value(N)

If a member is annotated `@value(N)`, the implementation SHALL additionally emit a numeric companion mapping named `<EnumName>Ordinal` so that the numeric value is recoverable:

```
enum Op {
  @value(0) ADD,
  @value(1) SUB,
  @value(7) NEG
};
```

maps to

```
export const Op = {
  ADD: "ADD", SUB: "SUB", NEG: "NEG",
} as const;
export type Op = (typeof Op)[keyof typeof Op];

export const OpOrdinal: Readonly<Record<Op, number>> = {
```



```

    ADD: 0, SUB: 1, NEG: 7,
  } as const;
  export const OpFromOrdinal: ReadonlyMap<number, Op> =
    new Map([[0, "ADD"], [1, "SUB"], [7, "NEG"]]);

```

If no member of the enum carries `@value(N)`, the implicit ordinals are 0,1,2,... in declaration order; the companion `<EnumName>Ordinal` object SHALL still be emitted with these implicit values.

7.6.3 @bit_bound(N)

`@bit_bound(N)` on an enum (XTypes 1.3 §7.2.2.4.7) SHALL be carried in the descriptor's `bitBound` field. The TypeScript surface is unaffected; this annotation governs the wire encoding only.

7.6.4 Enum Descriptor Side-Table

For every IDL `enum` the code generator SHALL emit a constant of type `DdsTypeDescriptor<EnumName>` with `kind: "enum"` plus the type-guard `isEnumName`. Each member SHALL appear as a `DdsMemberDescriptor` entry whose `type.kind = "primitive"` and whose default property carries the member's ordinal value. The descriptor's `bitBound` field SHALL carry the effective bit bound (default 32).

7.7 Bitset Mapping

An IDL `bitset` SHALL be mapped to a TypeScript `interface` with one property per bitfield, plus per-bitfield `export const <Bitset>_<Field>_BITS` declarations giving the width in bits as an integer literal. The TypeScript property type for a bitfield of width W SHALL be selected as follows:

- $W \leq 32$: TypeScript type `number`.
- W in 33..64: TypeScript type `bigint`.

This rule mirrors §7.8; bit operations on TypeScript `number` are coerced to Int32 and are unsafe for widths above 32 bits, while `bigint` preserves arithmetic for the full 64-bit IDL bitfield range.

```

bitset Flags {
  bitfield<3>   low;
  bitfield<5>   high;
  bitfield<40> wide;    // > 32 bits
};

```

maps to

```
export interface Flags {
  low: number;
  high: number;
  wide: bigint;
}
export const Flags_low_BITS = 3;
export const Flags_high_BITS = 5;
export const Flags_wide_BITS = 40;
```

The width-constant SHALL be a literal integer obtained by const-evaluation of the IDL `<positive_int_const>` expression in the bitfield declaration. Implementations MUST support decimal, hexadecimal, octal, and binary integer literal forms.

7.7.1 Bitset Descriptor Side-Table

For every IDL `bitset` the code generator SHALL emit a constant of type `DdsTypeDescriptor<Name>` with `kind: "bitset"`, plus a type-guard `isName`. Each bitfield SHALL appear as a `DdsMemberDescriptor` entry whose `type` is `{ kind: "bitfield", width: W }`. The descriptor's `bitBound` field SHALL carry the total width (sum of bitfield widths). This PSM SHALL reject bitsets whose total width exceeds 64; the IDL parser SHALL produce a fatal error in that case.

7.8 Bitmask Mapping

An IDL `bitmask` SHALL be mapped to a TypeScript `const` object whose entries are bit-shift expressions, plus a type alias. The element-type of the constant and of the alias SHALL be selected from the effective `@bit_bound` of the bitmask declaration:

- Effective `@bit_bound` ≤ 32 (default 32, OMG IDL 4.2 §7.4.13.5): TypeScript element type `number`; shift expressions SHALL use the unsigned 32-bit form `(1 << K) >> 0`.
- Effective `@bit_bound` in 33..64: TypeScript element type `bigint`; shift expressions SHALL use the BigInt form `1n << Kn`. The implementation MUST NOT emit `number`-typed shifts for $K \geq 32$, because the ECMAScript `<<` operator coerces both operands to `Int32` and would produce wrap-around values.

The two emission forms are mutually exclusive within a single bitmask declaration; mixed-width emission is forbidden. The declared bit width SHALL be exposed as `export const <Bitmask>_BIT_BOUND`.

```
bitmask Permissions { READ, WRITE, EXEC };
```

maps to

```
export const Permissions = {
  READ: (1 << 0) >>> 0,
  WRITE: (1 << 1) >>> 0,
  EXEC: (1 << 2) >>> 0,
} as const;
export type Permissions = number;
export const Permissions_BIT_BOUND = 32;
```

```
@bit_bound(48)
bitmask BigFlags {
  flag00, flag01, /* ... */ flag47
};
```

maps to

```
export const BigFlags = {
  flag00: 1n << 0n,
  flag01: 1n << 1n,
  // ...
  flag47: 1n << 47n,
} as const;
export type BigFlags = bigint;
export const BigFlags_BIT_BOUND = 48;
```

7.8.1 Bitmask Descriptor Side-Table

For every IDL `bitmask` the code generator SHALL emit a constant of type `DdsTypeDescriptor<Name>` with `kind: "bitmask"`, plus a type-guard `isName`. Each bit value SHALL appear as a `DdsMemberDescriptor` entry whose `default` property carries the bit position (0-based, per OMG IDL 4.2 §7.4.13.5) and whose `type` is `{ kind: "bitfield", width: 1 }`. The descriptor's `bitBound` field SHALL carry the effective bit bound.

7.8.2 Position Annotation

A bit value annotated with `@position(P)` (OMG IDL 4.2 §7.4.13.5) SHALL use `P` as the bit position in both the emitted shift expression and the corresponding `DdsMemberDescriptor.default` value. Conflicting positions (two bit values with the same explicit position) SHALL be rejected by the IDL parser.

7.9 Sequence and Array Mapping

7.9.1 Bounded and Unbounded Sequences

An IDL `sequence<T>` (unbounded) or `sequence<T, N>` (bounded by `N`) SHALL be mapped to a TypeScript `Array<T'>`, where `T'` is the TypeScript-mapped form of `T` per § 7.3.

The bound `N` of a bounded sequence SHALL be emitted as an exported numeric constant alongside the type declaration, named `<Container>_<Field>_BOUND`. This constant carries the upper limit and SHOULD be checked at the boundary of the runtime-validation layer.

```
struct Sample {
  sequence<long, 10>  pixels;    // bounded
  sequence<string>   tags;      // unbounded
};
```

maps to

```
export interface Sample {
  pixels: Array<number>;
  tags:   Array<string>;
}
export const Sample_pixels_BOUND = 10;
```

7.9.2 Fixed-Size Arrays

An IDL fixed-size array of `T` (`T name[N]`) SHALL be mapped to a TypeScript `Array<T'>` together with an exported numeric constant `<Container>_<Field>_LENGTH` that holds the array's exact length.

```
struct Vector3 { double v[3]; };
```

maps to

```
export interface Vector3 { v: Array<number>; }
export const Vector3_v_LENGTH = 3;
```

7.9.3 Multi-Dimensional Arrays

A multi-dimensional IDL array of dimensions `[N1][N2] ... [Nk]` SHALL be mapped to nested `Array` types of depth `k`. The length constants SHALL be emitted as `_LENGTH_DIM1`, `_LENGTH_DIM2`, ... suffixes preserving order of declaration.

7.9.4 Element Mutability

An implementation MAY emit `ReadonlyArray` in lieu of `Array` when (a) the containing struct or typedef carries an explicit `@final` extensibility annotation *and* (b) the implementation has been configured with `-readonly-collections`. Implementations SHALL NOT emit `ReadonlyArray` otherwise, since DDS data semantics permit in-place mutation between samples.

7.9.5 Empty Sequences

An empty IDL sequence-value SHALL round-trip through code generation as a TypeScript empty array (`[]`). Implementations MUST NOT encode an empty sequence as `undefined` or `null`; the property remains present with a zero-length array.

7.9.6 Maps

An IDL `map<K, V>` (unbounded) or `map<K, V, N>` (bounded by `N`), grammar (199) of OMG IDL 4.2 §7.4.13.4.3.1, SHALL be mapped according to the key type `K`:

- If `K` maps to a TypeScript `string` type (including `Char`, `WChar`, branded string aliases), the result SHALL be `ReadonlyMap<K', V'>` where `K'` and `V'` are the mapped forms of `K` and `V`. Implementations SHALL NOT use a TypeScript `Record<...>` for IDL maps because `Record` loses key-uniqueness validation and commits to enumeration order.
- If `K` maps to any other TypeScript type (`number`, `bigint`, `boolean`, struct, etc.), the result SHALL be `ReadonlyMap<K', V'>`.

The bound `N` of a bounded map SHALL be emitted as `export const <Container>_<Field>_BOUND` analogously to bounded sequences (§7.9).

The descriptor entry for a map field SHALL use a `DdsTypeRef` of the new kind `"map"`:

```
{ kind: "map"; key: DdsTypeRef; value: DdsTypeRef; bound?: number }
```

This kind SHALL be added to the `DdsTypeRef` union defined in Chapter 8.

```
struct Index {
  map<string<64>, long, 1024>  by_name;
};
```

maps to

```
export interface Index {
  by_name: ReadonlyMap<string, number>;
}
export const Index_by_name_BOUND = 1024;
```

An IDL key type that does not have value-equality semantics in JavaScript (e.g. a struct, which is compared by reference) SHALL be flagged with the descriptor field `keyEqualityHazard: true` on the corresponding `DdsMemberDescriptor`; consumers SHOULD use the runtime helper `equalKey(a, b, keyDe` for lookup rather than `Map.get` alone.

Construction. A producer constructs the map as a mutable `Map<K', V'>` and assigns it to the `ReadOnlyMap<K', V'>` property; this is covariantly sound in TypeScript and is the recommended pattern. An implementation MAY provide a `-mutable-collections` configuration flag (analogous to `-readonly-collections` for sequences, § 7.9) that emits `Map<K', V'>` instead of `ReadOnlyMap`; the flag is non-default.

7.10 Typedef Mapping

7.10.1 Simple Type Aliases

An IDL `typedef T name;` SHALL be mapped to a TypeScript `export type name = T';` where `T'` is the TypeScript-mapped form of `T`.

```
typedef long Counter;
typedef string<128> TopicName;
```

maps to

```
export type Counter = number;
export type TopicName = string;
```

7.10.2 Sequence Aliases

A typedef whose right-hand side is a sequence SHALL produce a TypeScript `Array`-aliased type. The bound constant of a bounded sequence SHALL be emitted as `<Alias>_BOUND`.

```
typedef sequence<octet, 1024> ByteBuffer;
```

maps to

```
export type ByteBuffer = Array<number>;
export const ByteBuffer_BOUND = 1024;
```

7.10.3 Array Aliases

A typedef whose right-hand side is a fixed-size array SHALL produce a TypeScript **Array**-aliased type with a corresponding `_LENGTH` constant.

7.10.4 Recursive Resolution

When a typedef chain contains multiple levels (`typedef A B; typedef B C;`), an implementation SHALL emit each declaration in source order. Each alias SHALL reference its immediate predecessor by name; cycles SHALL be rejected at IDL-parse time and SHALL NOT reach the code generator.

7.10.5 Forward References

If a typedef references a type declared later in the same compilation unit, the implementation SHALL emit all **export type** aliases and **export interface** declarations in any order: TypeScript resolves both interface and type-alias names at module scope independent of source-line order. The implementation SHALL NOT emit a forward declaration for a type alias; type aliases cannot be merged.

For **interface** declarations the same rule holds. When two interfaces refer to each other (mutual recursion), the implementation SHALL emit them in any order without synthesised forward stubs.

7.10.6 Typedef Descriptor Side-Table

For every named IDL typedef the code generator SHALL emit a constant of type `DdsTypeDescriptor<AliasName>` with `kind: "alias"` and a single `DdsMemberDescriptor` entry named `value` whose `type` carries the underlying `DdsTypeRef`. A type-guard `is<AliasName>` SHALL be emitted that delegates to the referenced type's guard.

7.10.7 @bit_bound on Integer Typedefs

An integer-valued typedef carrying `@bit_bound(N)` (XTypes 1.3 §7.2.2.4.7) SHALL be mapped according to the effective bound:

- $N \leq 32$: TypeScript surface unchanged (`number`); descriptor records `bitBound: N`.
- N in 33..64: TypeScript surface SHALL switch to `bigint`; descriptor records `bitBound: N`.

```
@bit_bound(40) typedef long long Counter40;
```

maps to

```
export type Counter40 = bigint;
export const Counter40Type: DdsTypeDescriptor<Counter40> = {
  kind: "alias", name: "Counter40",
  extensibility: "appendable", nested: false,
  bitBound: 40,
  fields: [{ name: "value", id: 0,
    type: { kind: "primitive", name: "int64" },
    key: false, optional: false, mustUnderstand: false }],
  typeGuard: isCounter40,
};
```

7.11 Optional and Default Values

7.11.1 Optional Fields

A field annotated `@optional` SHALL be mapped to a TypeScript property using the TypeScript `?:` property- suffix syntax; the property's declared type SHALL be `T' | undefined`, where `T'` is the TypeScript-mapped form of the IDL field type. The descriptor's corresponding `DdsMemberDescriptor` SHALL have `optional: true`.

```
@final struct Frame {
  long      seq;
  @optional
  long      retry;
};
```

maps to

```
export interface Frame {
  seq:      number;
  retry?: number | undefined;
}

export const FrameType: DdsTypeDescriptor<Frame> = {
  kind: "struct", name: "Frame",
  extensibility: "final", nested: false,
  fields: [
    { name: "seq", id: 0, type: { kind: "primitive", name: "int32" },
      key: false, optional: false, mustUnderstand: false },
    { name: "retry", id: 1, type: { kind: "primitive", name: "int32" },
      key: false, optional: true, mustUnderstand: false },
  ],
  typeGuard: isFrame,
};
```


7.11.2 Encoding of Absent Values (Informative)

The wire encoding of an absent optional value is governed by the encoding profile selected by the runtime, not by this specification. The following table is provided for cross-reference only and SHALL NOT be read as a normative requirement of this document; the authoritative source is the cited encoding specification.

Encoding	Absent-Value Form (Cross-Reference)
XCDR2	<code>is_present=0</code> octet preceding the value slot (OMG X-Types 1.3 §7.4.3).
JSON	The property key is omitted from the JSON object (DDS-JSON profile).
AMQP-Native	The field is encoded as the AMQP <code>null</code> primitive (DDS-AMQP §3.x).

What is *normative* in the present specification is the TypeScript-side representation: an absent optional value SHALL be modelled as the JavaScript `undefined` literal (not `null` and not a sentinel object). This is the value a Codegen-conformant consumer observes after a take/read.

7.11.3 Default Values

An IDL `@default(V)` annotation SHALL be carried in two places, neither of which alters the interface shape:

1. The corresponding `DdsMemberDescriptor` entry's `default` property SHALL hold the literal value `V`.
2. A side-table constant `export const <Type>_<Field>_DEFAULT` SHALL be emitted, typed identically to the field and initialised to the literal value. Consumer code that wishes to populate a default-bearing struct can use the constant directly.

A consumer-side helper `withDefaults<T>(descriptor: DdsTypeDescriptor<T>, partial: Partial<T>): T` SHOULD be exposed by the descriptor runtime; it builds a fully-populated `T` by filling absent properties from the descriptor's `default` metadata.

```
struct Config {
  @default(5000)    long    timeout_ms;
  @default("on")   string  mode;
};
```

maps to

```
export interface Config {
  timeout_ms: number;
```

```

    mode:          string;
}

export const Config_timeout_ms_DEFAULT: number = 5000;
export const Config_mode_DEFAULT:          string = "on";

export const ConfigType: DdsTypeDescriptor<Config> = {
  kind: "struct", name: "Config",
  extensibility: "appendable", nested: false,
  fields: [
    { name: "timeout_ms", id: 0, type: { kind: "primitive", name: "
      int32" }, key: false, optional: false, mustUnderstand: false
      , default: 5000 },
    { name: "mode",          id: 1, type: { kind: "string",      bound:
      undefined, wide: false }, key: false, optional: false,
      mustUnderstand: false, default: "on" },
  ],
  typeGuard: isConfig,
};

```

7.12 Annotation Mapping

7.12.1 Mapping Surface

This PSM does *not* emit any TypeScript class decorator, and does *not* require the `experimentalDecorators` TypeScript compiler option. Every IDL annotation is mapped to one or more of:

- a field of the side-table `DdsTypeDescriptor` or `DdsMemberDescriptor` (Chapter 8),
- a side-table `export const` (e.g. a default value constant),
- a TSDoc tag emitted immediately above the corresponding declaration.

7.12.2 Mapping Table

The following table is normative. An IDL annotation that parses but is not listed SHALL cause the code generator to emit DDS-TS-W002 and drop the annotation; under the `-strict-annotations` flag (which an implementation MUST provide), the same condition fires DDS-TS-E004 and aborts code generation instead.

IDL Annotation	TypeScript Mapping (Descriptor / TSDoc / Const)
@id(N)	DdsMemberDescriptor.id = N; TSDoc @dds-id N on the property.
@key	DdsMemberDescriptor.key = true; TSDoc @dds-key on the property; runtime helper <code>getKey</code> composes the key in declaration order.

IDL Annotation	TypeScript Mapping (Descriptor / TSDoc / Const)
@optional	TypeScript property declared with <code>?:</code> syntax and type <code>T undefined</code> (§ 7.11); <code>DdsMemberDescriptor.optional = true</code> .
@default(V)	Side-table export const <code><Type>_<Field>_DEFAULT</code> ; <code>DdsMemberDescriptor.default = V</code> (§ 7.11).
@final	<code>DdsTypeDescriptor.extensibility = "final"</code> .
@appendable	<code>DdsTypeDescriptor.extensibility = "appendable"</code> (implicit default if no extensibility annotation).
@mutable	<code>DdsTypeDescriptor.extensibility = "mutable"</code> ; every member MUST carry <code>@id(N)</code> .
@nested	<code>DdsTypeDescriptor.nested = true</code> ; TSDoc <code>@dds-nested</code> on the type.
@topic("name")	<code>DdsTypeDescriptor.topic = "name"</code> ; TSDoc <code>@dds-topic "name"</code> .
@must_understand	<code>DdsMemberDescriptor.mustUnderstand = true</code> ; TSDoc <code>@dds-must-understand</code> .
@unit("...")	<code>DdsMemberDescriptor.unit = "..."</code> ; TSDoc <code>@dds-unit</code> tag.
@min(V) / @max(V)	<code>DdsMemberDescriptor.min / .max = V</code> ; TSDoc <code>@dds-min / @dds-max</code> .
@hashid(...)	<code>DdsMemberDescriptor.hashid = ...</code> ; TSDoc <code>@dds-hashid</code> .
@autoid(SEQUENTIAL HASH)	<code>DdsTypeDescriptor.autoid = "sequential" "hash"</code> ; member ids assigned per the descriptor rule before emission.
@verbatim(...)	verbatim text emitted at the placement-kind position (Chapter 9).
@bit_bound(N)	numeric / bigint election in bitmask and bitset mappings (§ 7.8, § 7.7); recorded as <code>DdsTypeDescriptor.bitBound = N</code> .

7.12.3 Annotation Resolution Order

When multiple annotations apply to the same declaration, an implementation SHALL resolve them in the following order:

1. Extensibility (`@final`, `@appendable`, `@mutable`) at most one per type.
2. Identity (`@id`, `@autoid`) required on every member of a `@mutable` type.
3. Cardinality (`@optional`, `@key`) applied per member.
4. Validation and metadata (`@min`, `@max`, `@unit`, `@nested`, `@must_understand`, `@hashid`, `@topic`) recorded in the descriptor and rendered as TSDoc tags.

5. Verbatim (`@verbatim`) emitted last, possibly repeated, per Chapter 9.

A conflicting combination SHALL cause the IDL parser to reject the input with a fatal error; the code generator SHALL NOT encounter such a combination. Conflicts include at minimum:

- More than one extensibility annotation (`@final`, `@appendable`, `@mutable`) on a single type.
- Both `@autoid` on a type and `@id(N)` on any of its members: the explicit member id wins, but the parser SHALL require all members to have an explicit `@id` or none, never a mix.
- Two members of a single type sharing the same explicit `@id(N)`.
- Any member declared with `@id(0xFFFFFFFF)`: this id is reserved for the synthetic union discriminator (§7.5.3).
- Two bit values of a single `bitmask` sharing the same explicit `@position(P)`.
- Two cases of a single `union` sharing the same case label.

7.12.4 TSDoc Rendering

For every annotation that has a TSDoc tag in the table above, the implementation SHALL render the tag in a TSDoc block immediately preceding the corresponding TypeScript declaration. This permits TypeDoc-style documentation generators to surface the metadata without reparsing the IDL.

```
struct Telemetry {
    @unit("celsius") @min(-40) @max(125)
    double temperature;
};
```

maps to

```
export interface Telemetry {
    /**
     * @dds-unit celsius
     * @dds-min -40
     * @dds-max 125
     */
    temperature: number;
}
```

plus the descriptor entry recording `unit: "celsius", min: -40, max: 125`.

8 Descriptor Runtime (Normative)

This chapter defines the runtime surface that an IDL-Codegen- Profile implementation imports from and a Descriptor-Runtime- Profile implementation provides. No TypeScript class decorator is part of this PSM; reflection is provided exclusively through first-class descriptor values.

Runtime Package Name. Throughout this specification the runtime package is referenced as `@zerodds/types`. This identifier is the ZeroDDS reference name and is *vendor-specific*; another conformant vendor MAY publish the runtime under a different scope or unscoped name. A conformant implementation SHALL export every symbol defined in this chapter under exactly the names given (no prefixing, no renaming, no namespacing); only the import path is implementation-defined. Generated code emitted by a conformant codegen MAY therefore use any package path supplied via the `-runtime-package` configuration option, defaulting to `@zerodds/types`.

8.1 Extensibility Kind

```
export type ExtensibilityKind = "final" | "appendable" | "mutable";
```

The three values correspond directly to OMG X-Types 1.3 §7.2.2.4.4 `ExtensibilityKind` {FINAL, APPENDABLE, MUTABLE}.

8.2 Type References

A `DdsTypeRef` describes the type of a member in side-table form, sufficient for cross-vendor `TypeObject` construction.

```
export type PrimitiveName =
  | "boolean" | "octet"
  | "int16" | "uint16" | "int32" | "uint32"
  | "int64" | "uint64"
  | "float" | "double" | "longDouble"
  | "char" | "wchar";

export type DdsTypeRef =
  | { kind: "primitive"; name: PrimitiveName }
```

```

| { kind: "bitfield"; width: number } // 1..64; bitset / bitmask
  members only
| { kind: "string"; bound?: number; wide: boolean }
| { kind: "sequence"; element: DdsTypeRef; bound?: number }
| { kind: "array"; element: DdsTypeRef; lengths: ReadonlyArray<
  number> }
| { kind: "map"; key: DdsTypeRef; value: DdsTypeRef; bound?:
  number }
| { kind: "ref"; name: string } // qualified IDL name;
  resolve via lookupType
| { kind: "any" };

```

The `ref` kind carries the qualified IDL name of the referenced type rather than a direct `DdsTypeDescriptor` reference. Resolution is performed at use time via `lookupType(name)` (§8.7). This rule is normative and serves three goals: it permits mutually-recursive struct/union references that cannot be expressed in source-line-order initialisation, it makes a `DdsTypeRef` graph JSON-roundtrippable (no cycles in the value graph), and it aligns with the `TypeIdentifier` form used by OMG X-Types 1.3 for type lookup.

The `bitfield` kind is used exclusively for bitset member-references and bitmask bit-value references (§7.7, §7.8); `width` carries the bit count (1..64). Bit-bearing members SHALL NOT use `kind: "primitive"` since IDL has no `intN` primitive for arbitrary widths.

8.3 Type and Member Descriptors

```

export interface DdsMemberDescriptor {
  readonly name: string;
  readonly id: number;
  readonly type: DdsTypeRef;
  readonly key: boolean;
  readonly optional: boolean;
  readonly mustUnderstand: boolean;
  readonly default?: unknown;
  readonly unit?: string;
  readonly min?: unknown;
  readonly max?: unknown;
  readonly hashid?: string;
  // Union case-member only (qsec:union-descriptor).
  readonly labels?: ReadonlyArray<number | string | boolean>;
  // Map key with non-value-equality semantics (qsec:map-mapping).
  readonly keyEqualityHazard?: boolean;
}

export type DescriptorKind =
  | "struct" | "union" | "enum" | "bitset" | "bitmask"
  | "exception" | "alias" | "interface";

export interface DdsTypeDescriptor<T = unknown> {

```

```

readonly kind: DescriptorKind;
readonly name: string; // qualified IDL name
readonly extensibility: ExtensibilityKind;
readonly nested: boolean;
readonly topic?: string;
readonly autoid?: "sequential" | "hash";
readonly bitBound?: number;
// Union descriptors only (see §sec:union-descriptor).
readonly hasDefault?: boolean;
readonly hasImplicitDefault?: boolean;
readonly fields: ReadonlyArray<DdsMemberDescriptor>;
readonly typeGuard: (v: unknown) => v is T;
}

```

Exception Marker

```

export interface DdsException {
  readonly __dds_exception: true;
}

```

Every IDL exception interface (§7.4.3) SHALL extend `DdsException`.

The `kind` field disambiguates the descriptor's category; the `fields` array SHALL be empty for `enum`, `alias`, and primitive aliases. For interface descriptors the operation list is carried separately in a Service Descriptor (§11.5).

8.4 Boxed any

```

export interface DdsAny {
  readonly typeId: string; // qualified IDL name
  readonly value: unknown; // the boxed payload
}

```

8.5 Branded Character Helpers

The branded-type aliases `Char` and `WChar` together with their factory helpers `makeChar` and `makeWChar` are normatively defined in §7.3.2 and SHALL be exported from this package.

8.6 LongDouble

The opaque type `LongDouble` and its factory `makeLongDouble` are normatively defined in §7.3.3 and SHALL be exported from this package only when at least one consumer crate has been generated with `-preserve-long-double`; otherwise the symbol MAY be omitted.

8.7 Reflection API

```
export function registerType(d: DdsTypeDescriptor): void;
export function lookupType(qualifiedName: string): DdsTypeDescriptor |
    undefined;

export function getKey<T>(
    sample: T,
    descriptor: DdsTypeDescriptor<T>,
): ReadonlyArray<unknown>;

export function getTopic<T>(
    descriptor: DdsTypeDescriptor<T>,
): string | undefined;

export function withDefaults<T>(
    descriptor: DdsTypeDescriptor<T>,
    partial: Partial<T>,
): T;

export function boxAny<T>(
    descriptor: DdsTypeDescriptor<T>,
    value: T,
): DdsAny;

export function unboxAny<T>(
    any: DdsAny,
    descriptor: DdsTypeDescriptor<T>,
): T;

export function equalKey(
    a: unknown,
    b: unknown,
    keyType: DdsTypeRef,
): boolean;

export type GuardedTypeOf<D> =
    D extends DdsTypeDescriptor<infer T> ? T : never;

export function isOneOf<DS extends ReadonlyArray<DdsTypeDescriptor>>(
    err: unknown,
    descriptors: DS,
): err is Error & { cause: GuardedTypeOf<DS[number]> };
```


registerType. Adds the descriptor to a process-wide registry keyed by `DdsTypeDescriptor.name`. A re-registration with a structurally-equivalent descriptor SHALL be silently ignored; a re-registration with a structurally-different descriptor SHALL throw.

Structural equivalence is defined as deep value-equality of all descriptor fields except `typeGuard`, which is compared by reference identity (functions are not deep-equals comparable). Two code-gen outputs of the same IDL input by the same generator version SHALL produce structurally-equivalent descriptors with reference-identical `typeGuard` functions when the runtime is shared; cross- process or cross-bundle scenarios where two distinct `typeGuard` closures coexist are out of scope of the equivalence check (the registry sees only the first registration).

lookupType. Returns the previously-registered descriptor for a qualified name, or `undefined`.

getKey. Returns the values of all members flagged `key: true` in declaration order. The function SHALL NOT validate the sample's shape; the caller is expected to invoke `descriptor.typeGuard` first if they do not trust the sample's origin.

getTopic. Returns `descriptor.topic` unchanged.

withDefaults. Returns a new value of type T in which any property absent from `partial` but defined in the descriptor's member-list with a non-`undefined` `default` SHALL be filled from that default. Properties absent from both `partial` and the descriptor defaults remain absent.

boxAny. Constructs a `DdsAny` with `typeId=descriptor.name` and the supplied value. The function SHALL verify `descriptor.typeGuard(value)` and throw on failure.

unboxAny. Returns the typed value if and only if `any.typeId===descriptor.name` and `descriptor.typeGuard(any.value)` returns true; otherwise throws.

equalKey. Performs structural value equality between two key values described by `keyType`. For primitive, string, `bigint`, and branded character keys equality SHALL use JavaScript value-equality (equivalent to `===`) with one normative exception: `NaN`-valued floating-point keys SHALL compare equal to themselves (`Object.is`-style equality), and distinct `NaN`-bit-patterns SHALL compare as equal. `+0` and `-0` SHALL compare equal. For `ref`-kind keys referring to a struct, equality SHALL be performed recursively over every member of the referenced descriptor (overriding the JavaScript default of reference-equality on objects). For `any`-kind keys equality SHALL compare both `typeId` and the unboxed value via the matching descriptor. Map, sequence, and array values are not permitted as IDL keys.

Floating-point keys SHOULD be avoided in DDS topic designs because of representation drift; this PSM does not forbid them but documents the comparison rule for completeness.

isOneOf. Returns **true** if and only if **err** is an **Error** whose attached **cause** property satisfies the **typeGuard** of any of the supplied exception descriptors. Used in catch-handlers of generated client methods (§ 11.4). The implementation SHALL treat a non-**Error** **err** as **false**, never throw.

9 Verbatim Code-Generation Hook

9.1 Purpose

The X-Types 1.3 `@verbatim` annotation (*OMG X-Types* §7.2.2.4.8) provides an escape hatch by which IDL authors can inject language-specific source text into the generated output. This chapter normatively specifies how an IDL-Codegen-Profile implementation SHALL honour that annotation when the target language is TypeScript.

9.2 Annotation Form

The X-Types annotation form is:

```
@verbatim(language = "ts" | "*",
           placement = <PlacementKind>,
           text      = "<source-text>")
```

An implementation SHALL interpret the annotation when `language` is the literal string `"ts"` or the wildcard `"*"`. Other values SHALL cause the annotation to be ignored at TypeScript-codegen time (other language PSMs may handle them).

9.3 Placement Kinds

X-Types 1.3 §7.2.2.4.8 defines six placement kinds. An IDL-Codegen-Profile implementation SHALL support all six.

Placement Kind	TypeScript Emission Position
BEGIN_FILE	first lines of the generated <code>.ts</code> file, before the leading <code>// Generated by ...</code> -banner. Suited for license headers.
BEFORE_DECLARATION	immediately preceding the affected declaration, before any TSDoc comment.
BEGIN_DECLARATION	inside the declaration body, immediately after the opening brace. Used to inject helper methods on a class declaration.
END_DECLARATION	inside the declaration body, immediately before the closing brace.

Placement Kind	TypeScript Emission Position
AFTER_DECLARATION	immediately following the affected declaration.
END_FILE	last lines of the generated .ts file, after all declarations. Suited for default-exports or re-exports.

9.4 Quoting and Escaping

The verbatim `text` field is a string literal in IDL. The implementation SHALL resolve IDL-string-escape sequences (`\n`, `\t`, `\"`, `\\`) *before* emission, so that the resulting TypeScript source contains the unescaped text. Implementations SHALL NOT perform any TypeScript-specific re-escaping; the responsibility for valid TypeScript syntax lies with the IDL author.

9.5 Multi-Line Text

Multi-line verbatim blocks SHALL be emitted with a leading newline and a trailing newline, so that they integrate cleanly between adjacent declarations. Implementations MAY preserve indentation as written; they SHALL NOT alter inner whitespace.

9.6 Multiple Annotations on One Declaration

When several `@verbatim` annotations apply to the same declaration with the same placement-kind, they SHALL be emitted in source order. The implementation SHALL NOT re-order them or coalesce them.

9.7 Reference Implementation Footprint

The reference implementation `crates/idl-ts/src/verbatim.rs::emit_verbatim` exhausts all six placement kinds and is exercised by the test-fixture set `verbatim::tests::*`.

```
@verbatim(language = "ts",
           placement = "BEGIN_FILE",
           text      = "// Copyright Acme Corp.\n// SPDX-License-
                       Identifier: Apache-2.0")
struct AcmeMessage { long seq; };
```

produces

```
// Copyright Acme Corp.
// SPDX-License-Identifier: Apache-2.0
```

```
// Generated by zerodds idl-ts. Do not edit.  
  
export interface AcmeMessage { seq: number; }
```

10 Code Style (Informative)

This chapter is informative. It collects recommendations that do not affect conformance. Normative module and constant mapping rules are given in Chapter 7, § 7.1 and § 7.2.

10.1 Identifier Casing

By default the code generator preserves IDL identifier casing verbatim. A `-camelcase` configuration option may re-case `snake_case` IDL fields to `camelCase` TypeScript fields.

10.2 Output Format

The generated TypeScript source uses ECMAScript-Module (ESM) syntax. A `tsconfig.json` `target` of ES2022 or later is recommended; the generated source compiles cleanly under `tsc -strict`.

10.3 File Layout

Implementations may emit a single `.ts` file per IDL input file (default), one file per top-level module (`-module-per-file`, see § 7.1), or a custom layout driven by build-tooling integrations. None of these choices affects the conformance status of the generated code.

11 Interface and Operation Mapping (Normative)

This chapter is normative within the Operations Profile (§ 2.3). Implementations that do not claim Operations-Profile conformance MAY skip emission of the constructs defined here.

11.1 Mapping Surface

An IDL **interface** declaration carries operations (grammar (82) `<op_dcl>`, OMG IDL 4.2 §7.4.3.4.3.2) and attributes ((88) `<attr_dcl>`). Each interface SHALL be mapped to two TypeScript interface declarations plus one Service Descriptor:

1. A *client* interface, suffixed `<IfaceName>Client`, exposing each IDL operation and attribute as an asynchronous TypeScript method.
2. A *handler* interface, suffixed `<IfaceName>Handler`, exposing the same set of methods. A service-side implementer SHALL satisfy the handler interface; the framework delivers calls into it.
3. A constant of type `ServiceDescriptor<<IfaceName>Client, <IfaceName>Handler>` named `<IfaceName>Service`, carrying the operation metadata (§ 11.5).

No TypeScript class is emitted for an IDL interface; both client and handler are structural interfaces. Concrete client construction is the responsibility of the runtime (e.g. a DDS-RPC requester adapter), which produces objects satisfying the client interface from the Service Descriptor.

Operations SHALL be mapped to `Promise<T>`, not to synchronous returns. This diverges from the OMG C++/Java/C# PSMs and is necessary because TypeScript runs on event-loop runtimes where synchronous network I/O is unavailable. A handler that completes work synchronously SHALL still expose `Promise<T>` (returning via `Promise.resolve`).

11.2 Operation Mapping

For each IDL operation `<op_type> <name>(<params>) [raises (<exns>)]`; the generator SHALL emit a method whose:

- return type is `Promise<R>`, where `R` is the TypeScript-mapped form of `<op_type>` per Chapter 7, and `Promise<void>` for void-returning operations;

- parameter list is composed of the IDL `in` and `inout` parameters, in declaration order, each typed per Chapter 7; `out` parameters SHALL be aggregated into the resolved return value as described below;
- method name is the IDL operation name preserved verbatim, unless the `-camelcase` option is in effect.

Out and InOut parameters. If an operation declares either `out` or `inout` parameters, the resolved return value SHALL be an object literal carrying the declared return value as property `result` and one property per `out`/`inout` parameter, named after the IDL parameter. If the operation’s IDL return type is `void`, the `result` property is omitted. This rule preserves multiple-return semantics in TypeScript, which lacks native out-parameter passing.

`inout` parameters SHALL appear in both the method’s parameter list (as input) and in the resolved return value’s object (as output). The two are independent values; the parameter passes the initial value, the `result` property carries the value updated by the handler.

```
interface Cursor {
  long advance(in long step, inout long position, out boolean
    wrapped);
};
```

maps to

```
export interface CursorClient {
  advance(step: number, position: number):
    Promise<{ result: number; position: number; wrapped: boolean
      }>;
}
```

```
interface Calculator {
  long add(in long a, in long b) raises (Overflow);
  long divmod(in long a, in long b, out long remainder);
  oneway void reset();
};
```

maps to (selected client surface)

```
export interface CalculatorClient {
  add(a: number, b: number): Promise<number>;
  divmod(a: number, b: number): Promise<{ result: number; remainder
    : number }>;
  reset(): Promise<void>; // oneway: see below
}
```


Oneway operations. An IDL oneway operation SHALL return `Promise<void>`. The promise SHALL resolve when the call has been handed off to the transport, not when the handler has executed. This corresponds to the OMG IDL 4.2 oneway semantics (no reply).

No `async` keyword in interfaces. TypeScript does not permit the `async` keyword on interface method signatures (`async` is an implementation marker, not a type marker). The generated client and handler interfaces SHALL NOT use `async` on method signatures; the asynchronous semantics are carried entirely by the `Promise<T>` return type. A handler implementation (class or object satisfying the handler interface) MAY use `async` on its method bodies.

11.3 Attribute Mapping

An IDL attribute `T name;` declaration SHALL be mapped to a getter and (unless `readonly`) a setter on both the client and the handler interface:

```
get_${\langle$name\rangle}(): Promise<T>;
set_${\langle$name\rangle}(value: T): Promise<void>;
```

For `readonly` attribute the setter SHALL NOT be emitted. `getraises` / `setraises` clauses contribute to the attribute's exception set in the descriptor as defined in § 11.5.

11.4 Exception Mapping

An IDL exception declaration is mapped per § 7.4.3: a structural interface plus a descriptor with `kind: "exception"`.

When an operation declares `raises (E1, ..., En)`, the framework on the client side SHALL reject the returned promise with an `Error`-derived value whose attached `cause` is one of the structural exceptions `E1..En`. The runtime helper `isOneOf<E>(err, [...])` SHALL be exported by `@zerodds/types` as a type guard for narrowing in catch-handlers.

11.5 Service Descriptor

```
export type ParameterMode = "in" | "out" | "inout";

export interface OperationParameterDescriptor {
  readonly name: string;
  readonly mode: ParameterMode;
  readonly type: DdsTypeRef;
}

export interface OperationDescriptor {
  readonly name: string;
  readonly oneway: boolean;
```

```

    readonly returnType: DdsTypeRef | { kind: "void" };
    readonly parameters: ReadonlyArray<OperationParameterDescriptor>;
    readonly raises:      ReadonlyArray<DdsTypeDescriptor>;
  }

  export interface AttributeDescriptor {
    readonly name:      string;
    readonly readonly:  boolean;
    readonly type:      DdsTypeRef;
    readonly getRaises: ReadonlyArray<DdsTypeDescriptor>;
    readonly setRaises: ReadonlyArray<DdsTypeDescriptor>;
  }

  export interface ServiceDescriptor<TClient, THandler> {
    readonly name:      string;           // qualified IDL name
    readonly inherits:  ReadonlyArray<ServiceDescriptor<unknown, unknown>>;
    readonly operations: ReadonlyArray<OperationDescriptor>;
    readonly attributes: ReadonlyArray<AttributeDescriptor>;
  }

```

The implementation SHALL emit one `ServiceDescriptor` constant per IDL interface, named per the rule in §11.5 above. The descriptor's `inherits` array SHALL reflect the interface's inheritance specification (grammar (78) <interface_inheritance_spec>) in declaration order.

11.6 Inheritance

The TypeScript client and handler interfaces SHALL **extends** the corresponding mapped interfaces of every parent interface listed in the IDL inheritance specification. This propagates the operation set, attribute set, and (via `ServiceDescriptor.inherits`) the descriptor meta-data.

```

interface Base { long ping(); };
interface Counter : Base { long increment(); };

```

maps to

```

export interface BaseClient {
  ping(): Promise<number>;
}
export interface BaseHandler { ping(): Promise<number>; }

export interface CounterClient extends BaseClient {
  increment(): Promise<number>;
}
export interface CounterHandler extends BaseHandler {

```

```

    increment(): Promise<number>;
  }

  export const CounterService:
    ServiceDescriptor<CounterClient, CounterHandler> = {
      name:      "Counter",
      inherits:  [BaseService],
      operations: [/* increment only */,
      attributes: [],
    };

```

Multiple Inheritance. If an IDL interface lists more than one parent (`interface I : A, B`), the generated client and handler SHALL **extends** all parent client / handler interfaces in declaration order: `export interface IClient extends AClient, BClient { ... }`. TypeScript inherent rules apply: a method present on more than one parent with conflicting signatures SHALL cause a `tsc -strict` compile-time error; the IDL parser SHOULD pre-empt this by rejecting incompatible multiple-inheritance.

The descriptor's `inherits` array SHALL contain all parent service descriptors in declaration order; no flattening of inherited operations into the child descriptor is performed by this PSM.

11.7 Forward Declarations

An IDL `interface` forward-declaration (grammar (75) `<interface_forward_dcl>`) SHALL NOT produce a separate TypeScript declaration. Because TypeScript resolves type names at module scope independent of source-line order, the code generator SHALL emit only the complete declarations of every interface; declaration order in the output is implementation-defined and SHALL have no observable semantic effect.

If the IDL input contains a forward declaration without a matching complete declaration in the same compilation unit, the code generator SHALL emit a fatal error (DDS-TS-E002, “forward-declared interface lacks complete declaration”).

Annex A

(informative)

IDL Examples and Generated Output

A.1 Bounded Sequence

```
struct Telemetry {  
    string<32>          name;  
    sequence<double, 16> readings;  
};
```

maps to

```
export interface Telemetry {  
    name:      string;  
    readings: Array<number>;  
}  
export const Telemetry_name_BOUND = 32;  
export const Telemetry_readings_BOUND = 16;
```

A.2 Mutable Type with Explicit Ids

```
@mutable  
struct Config {  
    @id(0) string endpoint;  
    @id(1) long  timeout_ms;  
};
```

maps to

```
import {  
    DdsTypeDescriptor, registerType,  
} from "@zerodds/types";  
  
export interface Config {  
    endpoint: string;  
    timeout_ms: number;  
}  
  
export function isConfig(v: unknown): v is Config {
```

```

    return typeof v === "object" && v !== null
        && typeof (v as any).endpoint === "string"
        && typeof (v as any).timeout_ms === "number";
}

export const ConfigType: DdsTypeDescriptor<Config> = {
    kind: "struct", name: "Config",
    extensibility: "mutable", nested: false,
    fields: [
        { name: "endpoint", id: 0,
          type: { kind: "string", wide: false },
          key: false, optional: false, mustUnderstand: false },
        { name: "timeout_ms", id: 1,
          type: { kind: "primitive", name: "int32" },
          key: false, optional: false, mustUnderstand: false },
    ],
    typeGuard: isConfig,
};

registerType(ConfigType);

```

A.3 Bitset with Width-Constants

```

bitset RegisterMap {
    bitfield<8>    device_id;
    bitfield<3>    status;
    bitfield<5>    reserved;
};

```

maps to

```

export interface RegisterMap {
    device_id: number;
    status:    number;
    reserved:  number;
}
export const RegisterMap_device_id_BITS = 8;
export const RegisterMap_status_BITS    = 3;
export const RegisterMap_reserved_BITS  = 5;

```

Annex B

(normative)

Compliance Test Suite

B.1 Codegen-Profile Tests

1. **B.1.1** `boolean`, `octet`, `short`, `long`, `long long`, `float`, `double`, `string` round-trip through code generation and produce TypeScript that compiles under `tsc -strict` without `experimentalDecorators`.
2. **B.1.2** A `struct` with three fields produces an `export interface` with three properties, each typed per § 7.3, accompanied by a `<Type>Type: DdsTypeDescriptor` constant and an `is<Type>` guard per § 7.4. No `class` keyword MUST appear in the generator output for any IDL data type.
3. **B.1.3** A `union` produces a discriminated union with literal-narrowed `discriminator` properties per case (numeric, boolean, char or enum labels); a `default` case maps to an `Exclude<...>` arm; a generated `<Name>Type` descriptor with `kind: "union"` carries every case label in the `labels` property and the `hasDefault` / `hasImplicitDefault` flags per § 7.5.3.
4. **B.1.4** An `enum` produces a TypeScript `const` object plus a string-literal-union type alias (no `enum` keyword in the output); an `<Enum>Ordinal` side-table is emitted whether or not `@value(N)` is used; a descriptor with `kind: "enum"` and `bitBound: 32` (default) is emitted per § 7.6.4.
5. **B.1.5a** A `bitset` produces an `interface` plus per-bitfield `_BITS` constants with literal integer values; bitfields of width ≤ 32 use TypeScript `number`, widths 33..64 use `bigint`; a descriptor with `kind: "bitset"` is emitted per § 7.7.1.
6. **B.1.5b** A `bitmask` with default `@bit_bound` produces a `const` object of unsigned 32-bit shift expressions `((1 << K) >> 0)` and a `number` alias; with `@bit_bound` in 33..64 produces `1n << Kn` expressions and a `bigint` alias; `@position(P)` annotations override implicit positions; a descriptor with `kind: "bitmask"` is emitted per § 7.8.1.
7. **B.1.6** A module containing `typedef`, `struct`, `enum`, `union`, `bitmask`, and `bitset` declarations produces a single `export namespace` of the same name with all members declared, per the rule in § 7.1. Nested modules produce nested namespaces; qualified IDL references resolve to dotted TypeScript paths.

8. **B.1.7** Free-standing IDL constants (`const long`, `const double`, `const string`, `const boolean`, `const long long`) produce `export const`-declarations with the correct TypeScript type and a literal value, per §7.2. `long long` constants emit a `bigint`-suffix literal.
9. **B.1.8** `char` and `wchar` produce the branded type aliases `Char` / `WChar` per §7.3.2. `makeChar` rejects strings whose code-unit length $\neq 1$ or whose code point $> 0xFF$. `makeWChar` accepts astral-plane Unicode scalars (single code point of length-2 UTF-16 surrogate pair) and rejects unpaired surrogates.
10. **B.1.9** An IDL `long double` declaration produces a field of TypeScript type `LongDouble` per §7.3.3; `LongDouble` MUST have exactly the normative shape `{ readonly __dds_brand: "long_double", readonly bytes: Uint8Array }` with `bytes.length === 16`. The byte order inside `bytes` is implementation-defined and SHALL NOT be tested by this conformance suite. The generator MUST NOT substitute `number` and MUST NOT abort code generation on this construct.
11. **B.1.10** An IDL `any` declaration produces a property of TypeScript type `DdsAny` per §7.3.4; type `any` or `unknown` MUST NOT appear in the generator output for an IDL `any` member.
12. **B.1.11** An IDL `struct` carrying `@key`, `@id`, `@final`, `@appendable`, `@mutable`, `@optional`, `@default`, or `@topic` produces the same shape of `export interface` as an un-annotated struct; annotation values surface only in the side-table descriptor and (where applicable) TSDoc tags. No TypeScript `class` is emitted for any IDL data type.
13. **B.1.12** A free-standing IDL `exception` declaration produces an `export interface` extending `DdsException` and a descriptor with `kind: "exception"` per §7.4.3.
14. **B.1.13** A named IDL `typedef` produces a TypeScript `export type` alias and a descriptor with `kind: "alias"` per §7.10.6; an integer `typedef` carrying `@bit_bound(N)` with $N \in 33..64$ switches the alias to `bigint` per §7.10.7.
15. **B.1.14** An IDL `module` containing a free-standing `const`, a `struct`, a `union` with `default`, an `enum` with `@value(N)`, a `bitset` mixing widths ≤ 32 and > 32 , a `bitmask` with `@bit_bound(40)`, and a `typedef` compiles cleanly under `tsc -strict` without `experimentalDecorators`.
16. **B.1.15** An unrecognised but syntactically-valid IDL annotation causes the generator to emit DDS-TS-W002 and to drop the annotation; code generation MUST NOT abort.
17. **B.1.16** An IDL `map<K, V>` field produces a TypeScript `ReadonlyMap<K', V'>` property; a bounded map produces an additional `<Container>_<Field>_BOUND` constant; the descriptor entry uses `type.kind: "map"` per §7.9.6. A struct-typed key sets `keyEqualityHazard: true`.

B.2 Descriptor-Runtime Tests

1. **B.2.1** The package `@zerodds/types` exports the descriptor types and helper functions defined in Chapter 8: `DdsTypeDescriptor`, `DdsMemberDescriptor`, `DdsTypeRef`, `DdsAny`, `DdsException`, `DescriptorKind`, `ExtensibilityKind`, `PrimitiveName`, `Char`, `WChar`,

`makeChar`, `makeWChar`, `registerType`, `lookupType`, `getKey`, `getTopic`, `withDefaults`, `boxAny`, `unboxAny`, `equalKey`, `isOneOf`.

2. **B.2.2** `lookupType("FQN")` returns the descriptor previously registered with that name, or undefined.
3. **B.2.3** `getKey(sample, descriptor)` returns an array containing the values of all `key: true` members in declaration order; for a struct without key members it returns `[]`.
4. **B.2.4** `boxAny(descriptor, value)` throws when `descriptor.typeGuard(value)` returns false; otherwise it returns `{ typeId: descriptor.name, value }`.
5. **B.2.5** `unboxAny(any, descriptor)` returns the typed value when both `any.typeId` and `descriptor.typeGuard(any.value)` match, and throws otherwise.
6. **B.2.6** `withDefaults(descriptor, partial)` fills absent properties from descriptor-recorded defaults; properties with no default remain absent.
7. **B.2.7** `makeWChar("\u{1F600}")` (the smiling-face emoji, two UTF-16 code units, one Unicode scalar) succeeds; `makeChar("")` succeeds (U+00DF, in ISO 8859-1); `makeChar("")` throws (U+20AC, outside ISO 8859-1).
8. **B.2.8** `equalKey` returns true for two struct-typed keys with deeply-equal members; returns false for two reference-equal but structurally-different keys only if the members differ.
9. **B.2.9** `isOneOf(err, [E1Type, E2Type])` returns true for a rejection produced by an Operations- Profile client when the underlying IDL operation raised E1 or E2; returns false otherwise; never throws on non-Error input.

B.3 Operations-Profile Tests

1. **B.3.1** An IDL interface `I` produces both `IClient` and `IHandler` TypeScript interface declarations and an `IService: ServiceDescriptor` constant. No `class` keyword MUST appear in the generator output for any IDL interface.
2. **B.3.2** An operation `R foo(in T1 a, out T2 b)`; produces a method `foo(a: T1): Promise<{ result: R; b: T2 }>` on both client and handler.
3. **B.3.3** An operation `void bar(in T x) raises (E)`; produces a method `bar(x: T): Promise<void>` and the descriptor's `raises` list contains the `E` descriptor.
4. **B.3.4** A oneway `void baz()`; operation produces a method `baz(): Promise<void>` and the descriptor's `oneway` flag is true.
5. **B.3.5** A readonly attribute `T name`; produces `get_name(): Promise<T>` but no `set_name`.
6. **B.3.6** An interface inheritance chain `interface B : A {}` produces `BClient extends AClient`, `BHandler extends AHandler`, and `BService.inherits[0] === AService`.

7. **B.3.7** Multiple inheritance `interface I : A, B {}` produces `IClient extends AClient, BClient` and `IService.inherits` of length 2 in declaration order. A signature conflict between A and B on the same operation name SHALL cause `tsc -strict` to fail; the IDL parser SHOULD pre-empt this with a parser-level error.
8. **B.3.8** A forward-only interface declaration without a matching complete declaration in the same compilation unit causes DDS-TS-E002.
9. **B.3.9** A `readonly attribute T name` with `getraises (E)` produces a method `get_name(): Promise<T>` and the descriptor's `attributes[i].getRaises` contains the descriptor of E.

B.4 Reference Test-Harness

The reference test-harness is hosted at <https://gitlab.sandra-kessler.eu/fishermen21/zerodds/> under `crates/idl-ts/` (Rust unit tests against the codegen output) and verifies B.1, B.2, and (when the Operations Profile is claimed) B.3 by parsing the generated TypeScript with `tsc -noEmit -strict` and exercising the descriptor-runtime helpers in a Node.js test runner.

Annex C

(normative)

WASM-Bindings Profile

This annex normatively specifies the WASM-Bindings Profile of DDS-TS. An implementation that claims conformance to the Profile SHALL satisfy every requirement in this annex.

C.1 Handle and Sample Types

The handle and sample types referenced throughout this annex are normatively defined as follows. They SHALL be exported from the implementation's binding module under the names below.

C.1.1 Branded Handle Types

The handle types SHALL be defined using string-literal brand tags rather than module-local unique symbol declarations. String-literal brand tags compare structurally across module and vendor boundaries; unique symbol-based brands do not, and would prevent a handle produced by vendor A's binding from satisfying vendor B's handle parameter type even when the wire-level resource is identical.

```
export type ParticipantHandle = number & { readonly __dds_brand: "
  participant" };
export type TopicHandle      = number & { readonly __dds_brand: "topic
  " };
export type PublisherHandle  = number & { readonly __dds_brand: "
  publisher" };
export type SubscriberHandle = number & { readonly __dds_brand: "
  subscriber" };
export type DataWriterHandle = number & { readonly __dds_brand: "
  writer" };
export type DataReaderHandle = number & { readonly __dds_brand: "
  reader" };
```

The brand-tag values ("participant", "topic", "publisher", "subscriber", "writer", "reader") are normative; conformant implementations SHALL NOT substitute alternative tag values. The property-name `__dds_brand` is also normative. This guarantees that two conformant TypeScript bindings even from distinct vendors and distinct npm packages produce type-compatible handle types.

Each handle is a non-negative 32-bit integer index into the host’s resource table. Branding prevents accidental cross-assignment between handle kinds at compile time; the brand is a TypeScript-only marker without runtime effect. A handle value of 0 is reserved as the “invalid handle” sentinel this is a *runtime-enforced* invariant: the type system cannot prevent 0 as `DataReaderHandle` from compiling, so any operation receiving 0 SHALL reject with an error at runtime.

C.1.2 Sample Type

```

/** OMG DDSI-RTPS GUID (16 bytes = 12-byte GuidPrefix + 4-byte EntityId
    ). */
export interface DdsGuid {
  readonly prefix: Uint8Array; // length === 12
  readonly entityId: number; // 32-bit unsigned
}

export function makeDdsGuid(prefix: Uint8Array, entityId: number):
  DdsGuid {
  if (prefix.length !== 12) {
    throw new RangeError("DdsGuid.prefix requires 12 bytes");
  }
  if (!Number.isInteger(entityId) || entityId < 0 || entityId > 0
    xFFFFFFFF) {
    throw new RangeError("DdsGuid.entityId requires uint32");
  }
  return { prefix, entityId };
}

export interface SampleInfo {
  readonly validData: boolean;
  readonly sampleState: "read" | "not_read";
  readonly viewState: "new" | "not_new";
  readonly instanceState: "alive" | "not_alive_disposed" | "
    not_alive_no_writers";
  readonly sourceTimestampNs: bigint;
  readonly sequenceNumber: bigint;
  readonly publicationHandle: DdsGuid;
  readonly instanceHandle: bigint;
}

export interface Sample {
  readonly bytes: Uint8Array; // XCDR2-encoded payload
  readonly info: SampleInfo;
}

```

The `bytes` field carries the XCDR2 octet sequence exactly as it crosses the wire. The `info` object is a strict subset of the OMG DDS 1.4 `SampleInfo` structure (§2.2.2.5.4); fields not enumerated above are not exposed by this Profile. `publicationHandle` carries the full DDSI-RTPS GUID of the originating writer (12-byte `GuidPrefix` carrying vendor-id, host-id, app-id,

and instance-id components per OMG DDSI-RTPS 2.5 §8.2.4, plus 4-byte `EntityId`); truncation to a 64-bit subset SHALL NOT be performed because it loses the vendor/host components and risks hash collisions in multi-vendor or multi-host deployments. `instanceHandle` is opaque and remains a 64-bit handle.

C.1.3 Listener Callback Type

```
export type DataAvailableCallback = (r: DataReaderHandle) => void;
```

C.2 Required Operations

The WASM-Bindings Profile SHALL expose the following DDS DCPS data-path operations as TypeScript-callable functions. The signatures are given in TypeScript and are normative; the underlying WebAssembly export-name is implementation-defined.

C.2.1 Participant and Topic

```
function createParticipant(domainId: number): ParticipantHandle;
function deleteParticipant(p: ParticipantHandle): void;

function createTopic(p: ParticipantHandle,
                    name: string,
                    typeName: string): TopicHandle;
function deleteTopic(t: TopicHandle): void;
```

C.2.2 Publisher and Subscriber

```
function createPublisher (p: ParticipantHandle): PublisherHandle;
function createSubscriber(p: ParticipantHandle): SubscriberHandle;
function deletePublisher (pub: PublisherHandle): void;
function deleteSubscriber(sub: SubscriberHandle): void;
```

C.2.3 DataWriter and DataReader

```
function createDataWriter(pub: PublisherHandle,
                        topic: TopicHandle): DataWriterHandle;
function createDataReader(sub: SubscriberHandle,
                        topic: TopicHandle): DataReaderHandle;

function writeSample (w: DataWriterHandle, xcdr2: Uint8Array): void;
```

```
function takeSamples (r: DataReaderHandle, max: number): ReadonlyArray<
    Sample>;

function deleteDataWriter(w: DataWriterHandle): void;
function deleteDataReader(r: DataReaderHandle): void;
```

C.2.4 Listener Registration

```
function setDataAvailableListener(
    r: DataReaderHandle,
    cb: DataAvailableCallback | null,
): void;
```

A `cb` value of `null` SHALL unregister any previously-installed listener. Listener invocation SHALL occur on the host event loop, never inside a synchronous `writeSample` call.

C.3 Wire-Format Boundary

C.3.1 XCDR2 Sample Bytes

Sample data SHALL cross the WASM boundary as `Uint8Array` of XCDR2-serialised octets, encoded according to OMG X-Types 1.3 §7.4.3. The selected encapsulation kind (`CDR2_BE` or `CDR2_LE`) SHALL be configured via the codegen flag `-xcdr-endianness=BE|LE`, defaulting to `LE` (the wire-form predominant in practice). Implementations MUST NOT expose JavaScript-native object representations as the canonical wire form; the binding layer is purely a WebAssembly-glue layer.

C.3.2 Memory Ownership

The TypeScript caller SHALL retain ownership of any `Uint8Array` passed across the boundary; the WASM module SHALL treat such buffers as borrowed and SHALL NOT retain references after the call returns. For samples returned from `takeSamples`, the WASM module SHALL allocate fresh `Uint8Array` buffers owned by JavaScript.

C.3.3 DdsAny Across the Boundary

A topic-type that contains an IDL `any` field (§7.3.4) crosses the WASM boundary as a fully serialised XCDR2 payload exactly like any other topic-type; `DdsAny` is a TypeScript-side helper for narrowing on take and for boxing on write, not a separate wire object. The producer SHALL call `boxAny(...)` on the TypeScript side, then serialise the resulting compound topic through the codegen-emitted serialiser; the consumer SHALL deserialise the topic, then call `unboxAny(...)` for narrowing.

C.4 Browser vs Node.js Bindings

C.4.1 Browser Profile

A browser-side conformant implementation SHALL expose the operations of § C.2 via a single `.wasm` module compiled with `wasm-bindgen` or an equivalent toolchain. The underlying transport SHALL use WebTransport, WebSocket, or HTTP-3 for off-host communication; native UDP-multicast is not available in browsers and MUST NOT be required.

C.4.2 Node.js Profile

A Node.js-side conformant implementation MAY additionally provide a native `.node` module via N-API (`napi-rs`) for performance-sensitive deployments. The exposed API SHALL match § C.2 signature-for-signature, so a TypeScript application can be retargeted between browser and Node.js without source-code changes outside the import statement.

C.5 Conformance Tests

C.5.1 Round-Trip Publish/Subscribe

A WASM-Bindings-Profile implementation SHALL pass the following round-trip test:

1. Create a Participant in domain 0, a Topic "Echo" with type-name "PrimitiveSample", a DataWriter, and a DataReader on the Topic.
2. Encode a sample of an IDL struct `struct PrimitiveSample { long v; }` via the IDL-Codegen-Profile output and write the resulting XCDR2 bytes via `writeSample`.
3. Within 1s, `takeSamples` SHALL return an array of length 1 whose `info.validData` is `true` and whose decoded payload `v` equals the published value.

C.5.2 Listener Callback

After registering a `DataAvailable` listener, writing a sample SHALL cause the listener to be invoked at least once. Listener delivery SHALL NOT occur synchronously within `writeSample`.

C.5.3 Resource Cleanup

Calling `deleteParticipant` SHALL release all WebAssembly memory associated with the Participant; subsequent calls referencing any of its derived handles SHALL throw a `RangeError` naming the invalidated handle. A double-delete of any handle SHALL also throw, never silently succeed.

C.5.4 DdsAny Round-Trip

A topic containing a field of IDL type **any** SHALL round-trip via the producer/consumer pattern in § C.3.3 such that the consumer's **unboxAny** call produces a value equal to the original under **descriptor.typeGuard**.

C.6 Reservation of Future Extensions

The following identifiers are reserved for future revisions of this Profile and SHALL NOT be re-used by Beta implementations for unrelated purposes: **createGuardCondition**, **createWaitSet**, **createQueryCondition**, **getInstance**.

The identifier **registerType** is *not* reserved here: it is used by the Descriptor-Runtime Profile (§ 2.2) for its specified semantics (§ 8.7).

Annex D

(normative)

Diagnostic Codes

A conformant code generator SHALL use the following diagnostic codes when emitting the conditions enumerated below. The code, severity, and trigger condition are normative; the human-readable message text is implementation-defined and SHOULD cite the source location of the offending IDL construct.

Code	Severity	Trigger
DDS-TS-E001	fatal	<i>Reserved</i> for future use; conformant implementations SHALL NOT re-use this code.
DDS-TS-E002	fatal	A forward-declared interface (<interface_forward_dcl>) lacks a matching complete declaration in the same compilation unit (§ 11, Forward Declarations).
DDS-TS-E003	fatal	The IDL parser encountered a conflicting annotation combination as enumerated in § 7.12; the parser SHALL reject the input.
DDS-TS-E004	fatal	Under -strict-annotations , an unrecognised IDL annotation was encountered (else fires as DDS-TS-W002, § 7.12).
DDS-TS-W001	warning	<i>Reserved</i> for future use; conformant implementations SHALL NOT re-use this code.
DDS-TS-W002	warning	A syntactically-valid IDL annotation not listed in § 7.12 was encountered; the annotation is dropped, code generation continues.
DDS-TS-W003	warning	An IDL map key type with non-value-equality JavaScript semantics (§ 7.9.6); descriptor entry carries keyEqualityHazard: true .
DDS-TS-W004	warning	An IDL union with no default and non-exhaustive cases (§ 7.5); descriptor carries hasImplicitDefault: true .
DDS-TS-I001	info	Use of IDL long double ; mapped to the opaque LongDouble carrier (§ 7.3.3).

Codes E001..E099, W001..W099, and I001..I099 are reserved for this specification; codes E100+, W100+, I100+ are available for implementation-specific diagnostics. A diagnostic SHALL carry at minimum the code, severity, file path, source line and column, and message text; the surface format (terminal/LSP/JSON) is implementation-defined.